

Start coding or [generate](#) with AI.



Airline Customer Satisfaction Analysis: Random Forest Optimization

Objective: Engineer a scale-invariant Random Forest ensemble pipeline utilizing a strict three-way split partition and PredefinedSplit hyperparameter tuning to mitigate variance and maximize classification performance.

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV, PredefinedSplit
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# 1. Identify and ingest the airline satisfaction dataset file
target_file = None
for file in os.listdir('.'):
    if file.endswith('.csv') and '4bb936ec' not in file: # Exclude previous asset
        target_file = file
        break

if target_file:
    print(f>Loading data from resource file: {target_file}")
    df = pd.read_csv(target_file)
else:
    raise FileNotFoundError("Airline satisfaction CSV source asset could not be found")

print(f>Data Schema Initialized: {df.shape[0]} Passengers x {df.shape[1]} Features
```

```
Loading data from resource file: 966bb80f-b8a8-4a15-acd0-e5fa46ae7b6b.csv
Data Schema Initialized: 129880 Passengers x 22 Features
```

```
# 1. Encode target variable cleanly to binary integers
df['satisfaction'] = (df['satisfaction'] == 'satisfied').astype(int)

# 2. Impute missing numerical columns using median values to preserve feature distribution
if df.isnull().sum().sum() > 0:
    print("Missing fields identified. Executing median imputation strategy...")
    for col in df.select_dtypes(include=[np.number]).columns:
```

```
df[col] = df[col].fillna(df[col].median())
```

```
# 3. Identify categorical values for explicit encoding mapping
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
print(f"Categorical features targeted for encoding: {categorical_cols}")

# Transform categorical vectors via One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
X = df_encoded.drop(columns=['satisfaction'])
y = df_encoded['satisfaction']
```

Missing fields identified. Executing median imputation strategy...

Categorical features targeted for encoding: ['Customer Type', 'Type of Travel',

```
# 1. Isolate the final hold-out evaluation Test set (20% of total asset volume)
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

```
# 2. Slice the remaining data to separate pure Training data from the Validation Set
# Splitting train_val (80%) into 75% train and 25% val yields an exact 60/20/20 split
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=0.25, random_state=42, stratify=y_train_val
)
```

```
print(f"Split Layout Verification:")
print(f"└─ Pure Training Subset:    {X_train.shape[0]} records (60%)")
print(f"└─ Validation Tuning Set:    {X_val.shape[0]} records (20%)")
print(f"└─ Unseen Testing Set:       {X_test.shape[0]} records (20%)")
```

```
Split Layout Verification:
└─ Pure Training Subset:    77928 records (60%)
└─ Validation Tuning Set:    25976 records (20%)
└─ Unseen Testing Set:      25976 records (20%)
```

```
# To prevent default K-fold shuffling, we combine train and validation back together
X_cv_comb = pd.concat([X_train, X_val]).reset_index(drop=True)
y_cv_comb = pd.concat([y_train, y_val]).reset_index(drop=True)
```

```
# Build an array where:
# -1 flags indices belonging to the training set (ignored during parameter tuning)
# 0 flags indices belonging to the validation set (explicit evaluation split)
split_index = np.concatenate([
    np.full(X_train.shape[0], -1),
    np.full(X_val.shape[0], 0)
])
```

```
custom_split = PredefinedSplit(test_fold=split_index)
print("PredefinedSplit mask successfully mapped to isolate the validation block")
```

PredefinedSplit mask successfully mapped to isolate the validation block from tr

```
# Define hyperparameter grid matrix constraints
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [10, 15, None],
    'min_samples_leaf': [2, 4]
}

rf_base = RandomForestClassifier(random_state=42, n_jobs=-1)

# Initialize GridSearchCV tracking using our custom PredefinedSplit index frame
grid_search = GridSearchCV(
    estimator=rf_base,
    param_grid=param_grid,
    cv=custom_split,
    scoring='f1',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_cv_comb, y_cv_comb)

print(f"\nOptimal Parameter Configuration Identified:")
print(grid_search.best_params_)
best_rf_model = grid_search.best_estimator_
```

Fitting 1 folds for each of 12 candidates, totalling 12 fits

Optimal Parameter Configuration Identified:
{'max_depth': None, 'min_samples_leaf': 2, 'n_estimators': 100}

```
# Construct an overfit baseline model to illustrate the variance-reduction bene
dt_baseline = DecisionTreeClassifier(random_state=42)
dt_baseline.fit(X_train, y_train)

print("Baseline Decision Tree initialized and trained on baseline data slices.")
```

Baseline Decision Tree initialized and trained on baseline data slices.

```
# Execute test predictions across both model frameworks
y_pred_dt = dt_baseline.predict(X_test)
y_pred_rf = best_rf_model.predict(X_test)

# Compile performance data arrays
metrics_summary = {
    'Evaluation Metric': ['Accuracy', 'Precision', 'Recall', 'F1-Score'],
    'Baseline Decision Tree': [
        accuracy_score(y_test, y_pred_dt),
```

```
        precision_score(y_test, y_pred_dt),
        recall_score(y_test, y_pred_dt),
        f1_score(y_test, y_pred_dt)
    ],
    'Optimized Random Forest': [
        accuracy_score(y_test, y_pred_rf),
        precision_score(y_test, y_pred_rf),
        recall_score(y_test, y_pred_rf),
        f1_score(y_test, y_pred_rf)
    ]
}

performance_df = pd.DataFrame(metrics_summary)
print("=== AIRLINE EXECUTIVE MODEL PERFORMANCE METRIC MATRIX ===")
print(performance_df.to_string(index=False))

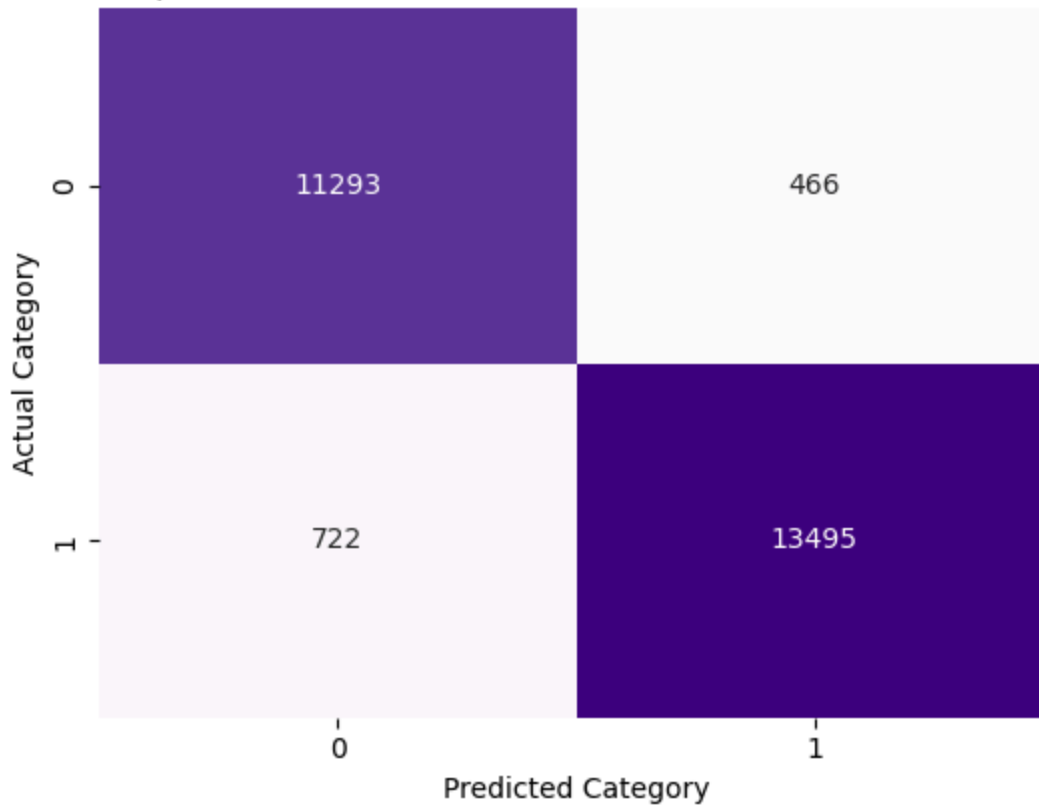
# Export and save confusion matrix visual artifact
cm = confusion_matrix(y_test, y_pred_rf)
plt.figure(figsize=(5.5, 4.5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples', cbar=False)
plt.title('Optimized Random Forest Classifier: Confusion Matrix')
plt.ylabel('Actual Category')
plt.xlabel('Predicted Category')
plt.tight_layout()
plt.savefig('rf_airline_confusion_matrix.png', dpi=300)
plt.show()
```

```

=== AIRLINE EXECUTIVE MODEL PERFORMANCE METRIC MATRIX ===
Evaluation Metric  Baseline  Decision Tree  Optimized Random Forest
Accuracy          0.928549      0.954265
Precision         0.933811      0.966621
Recall            0.935781      0.949216
F1-Score          0.934795      0.957839

```

Optimized Random Forest Classifier: Confusion Matrix



```

# Isolate feature importances from our best Random Forest estimator
importances = best_rf_model.feature_importances_
feature_names = X.columns

fi_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
fi_df = fi_df.sort_values(by='Importance', ascending=False).head(7)

# Generate horizontal feature prioritization profile chart
plt.figure(figsize=(8, 4.5))
sns.barplot(x='Importance', y='Feature', data=fi_df, palette='viridis', hue='Fe
plt.title('Top 7 Passenger Satisfaction Structural Optimization Drivers', fonts
plt.xlabel('Relative Feature Importance Contribution Score')
plt.tight_layout()
plt.savefig('rf_feature_importances.png', dpi=300)
plt.show()

```

